

Техническое задание: MPV CLI Workflow для ShortFarm

Контекст и цели

Проект **ShortFarm** предназначен для массовой нарезки видео на короткие фрагменты (примерно минутной длины), которые затем публикуются на различные платформы. Инструмент должен масштабироваться на большой объём видео, множество каналов и различные языки, при этом оставаясь локальным и работающим из терминала.

Главная цель данного ТЗ — добавить в существующий Python-проект workflow для ручной разметки видео через **mpv**. Пользователь сможет просматривать видео, отмечать нужные фрагменты горячими клавишами и отправлять их в очередь на рендер. Особое внимание уделяется удобству CLI, сохранению состояния, корректной обработке ошибок и будущей расширяемости.

Оценка текущего состояния

Проект уже имеет ясную структуру:

- `config.py` хранит настройки проекта.
- `db.py` содержит функции для работы с SQLite-базой.
- `services.py` инкапсулирует бизнес-логику.
- `cli.py` реализует интерфейс командной строки на базе Typer.
- Есть функции для добавления видео, split/cut и управление jobs.

Существующие возможности подходят для обработки видеофайлов и создания нарезок с помощью `ffmpeg`, но отсутствует удобный режим просмотра и отметки моментов вручную. Кроме того, в текущем виде отсутствуют миграции, некоторые функции дублируются, а состояние видео (просмотрено, пропущено, в работе) не хранится явно.

Основные принципы и архитектура

Минимальная функциональность (MVP)

1. **Посмотреть видео и отметить фрагменты:** использовать `mpv` как видеоплеер и Lua-скрипт для записи событий (начало, конец, быстрый 60-секундный клип, отмена, завершение, пропуск, выход).
2. **Импортировать разметку:** после закрытия `mpv` Python-код считывает JSONL-файл с событиями, добавляет метки в базу и формирует очередь клипов.
3. **Рендер клипов:** `ffmpeg` нарезает нужные отрезки, записывая выходные файлы через временные файлы, чтобы избежать повреждения.
4. **Сохранение состояния:** статус каждого видео (`inbox`, `reviewing`, `reviewed`, `skipped`, `failed`) хранится в базе, как и статусы клипов (`queued`, `rendering`, `done`, `failed`).

5. **Чёткие таблицы и миграции:** схема базы данных управляется через миграции; любые изменения схемы фиксируются.

Модульная структура

```
shortfarm/
├── __init__.py
├── __main__.py
├── config.py
├── db.py           # работа с SQLite и миграции
├── services.py     # бизнес-логика (импорт sessions, marks и clips)
├── ffmpeg_tools.py # вызовы ffmpeg/ffprobe
├── cli.py          # Typer CLI
├──
├── mpv_session.py  # запуск mpv, создание session, импорт JSONL
├── render.py        # обработка очереди клипов (ffmpeg нарезка)
├── migrations.py    # helpers для применения миграций
├──
└── scripts/
    └── shortfarm.lua # Lua-скрипт для mpv
```

Файлы Lua

Lua-скрипт `shortfarm.lua` должен находиться в папке `shortfarm/scripts`. Путь к нему в Python определяется через `Path(__file__).resolve().parent / "scripts" / "shortfarm.lua"`; на него нельзя полагаться как на текущую рабочую директорию. Скрипт должен поддерживать горячие клавиши и писать события в JSONL-файл.

База данных и миграции

- Все таблицы создаются через миграции, а факт их применения записывается в таблицу `schema_migrations(version, applied_at)`.
- При первом запуске проекта или `shortfarm init` необходимо создать эту таблицу, проверить существующие таблицы и зарегистрировать базовую версию.
- Любые изменения схемы должны идти в новых миграциях.

Таблицы в SQLite

videos

Таблица `videos` (существует в проекте) должна быть расширена колонкой `review_status`:

```
ALTER TABLE videos ADD COLUMN review_status TEXT NOT NULL DEFAULT 'inbox';
```

Статусы:

- `inbox` — видео ещё не просматривалось;
- `reviewing` — видео взято на просмотр в mpv;

- `reviewed` — просмотр завершён, отметки импортированы;
- `skipped` — пользователь решил пропустить видео;
- `failed` — просмотр не удался по техническим причинам.

review_sessions

Фиксирует каждый запуск mpv для конкретного видео и ссылку на файл с событиями:

```
CREATE TABLE review_sessions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  video_id INTEGER NOT NULL,
  session_file TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'open',
  started_at TEXT NOT NULL,
  finished_at TEXT,
  imported_at TEXT,
  error TEXT,
  FOREIGN KEY(video_id) REFERENCES videos(id)
);
```

Статусы `review_sessions`:

- `open` — создана запись, mpv запускается или работает;
- `closed` — mpv завершил работу, но импорт ещё не выполнен;
- `imported` — события импортированы;
- `failed` — mpv не запустился или импорт JSONL завершился с фатальной ошибкой;
- `abandoned` — пользователь явно сбросил сессию (через `review reset`).

marks

Хранит отмеченные фрагменты:

```
CREATE TABLE marks (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  video_id INTEGER NOT NULL,
  session_id INTEGER,
  in_sec REAL NOT NULL,
  out_sec REAL NOT NULL,
  rating INTEGER,
  label TEXT,
  source TEXT NOT NULL DEFAULT 'mpv',
  created_at TEXT NOT NULL,
  FOREIGN KEY(video_id) REFERENCES videos(id),
  FOREIGN KEY(session_id) REFERENCES review_sessions(id)
);
```

Условия:

- `out_sec > in_sec`;

- `rating` либо NULL, либо целое число в диапазоне 1...5.

clips

Очередь на нарезку и рендер:

```
CREATE TABLE clips (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  video_id INTEGER NOT NULL,
  mark_id INTEGER,
  status TEXT NOT NULL DEFAULT 'queued',
  cut_mode TEXT NOT NULL DEFAULT 'exact',
  output_path TEXT,
  temp_path TEXT,
  error TEXT,
  created_at TEXT NOT NULL,
  started_at TEXT,
  rendered_at TEXT,
  FOREIGN KEY(video_id) REFERENCES videos(id),
  FOREIGN KEY(mark_id) REFERENCES marks(id)
);
```

Статусы клипов:

- `queued` — ждёт рендер;
- `rendering` — процесс ffmpeg запущен;
- `done` — файл успешно создан;
- `failed` — рендер закончился ошибкой.

Миграции

Каждый SQL-файл должен соответствовать версии и иметь явное описание. Пример:

- 001_initial_existing_schema.sql — регистрация существующих таблиц.
- 002_add_review_status.sql — добавление review_status в videos.
- 003_create_review_sessions.sql — создание таблицы review_sessions.
- 004_create_marks.sql — создание таблицы marks.
- 005_create_clips.sql — создание таблицы clips.

CLI-команды

Ниже приведены команды, которые должны быть реализованы или сохранены.

Существующие команды (без изменений)

- `shortfarm init` — инициализация базы и миграций.
- `shortfarm doctor` — проверяет наличие mpv, ffmpeg/ffprobe, Lua-скрипта и состояние базы.
- `shortfarm add <path>` — добавляет файлы в базу `videos`.
- `shortfarm inbox` — показывает список видео со статусами и количеством меток/клипов.

- `shortfarm split` / `shortfarm cut` и другие существующие команды — остаются без изменений.

Команды для просмотра (review)

- `shortfarm review inbox` — атомарно выбирает одно видео со статусом `inbox`, переводит его в `reviewing`, создаёт `review_session` со статусом `open` и запускает `mpv`. Если подходящего видео нет, выводит сообщение "No inbox videos available".
- `shortfarm review open-id <video_id>` — открывает видео по ID. Если видео в статусе `inbox`, оно переводится в `reviewing`. Если статус `reviewing`, команда отказывает и предлагает воспользоваться `review reset`. Для статусов `reviewed`, `skipped`, `failed` команда отказывается без `--force`; флаг `--force` разрешает повторный просмотр завершённого видео.
- `shortfarm review open <path>` — добавляет файл в базу (если его нет) и обрабатывает аналогично `open-id`.
- `shortfarm marks <video_id>` — выводит метки видео.
- `shortfarm skip <video_id>` — помечает видео как `skipped`.
- `shortfarm review reset <video_id>` (или `--abandon-open-session`) — переводит `review_status` обратно в `inbox` и помечает открытые `review_sessions` как `abandoned`.

Команды для клипов и рендера

- `shortfarm clips [--status <status>] [--video-id <id>]` — показывает клипы по статусу или по конкретному видео.
- `shortfarm render [--limit <n>]` — берёт первые `n` клипов со статусом `queued`, переводит их в `rendering`, запускает `ffmpeg`. По умолчанию рендер точный (`cut_mode=exact`).
- `shortfarm render-all` — запускает рендер для всей очереди.
- `shortfarm retry-failed [--clip-id <id>]` — переводит `failed` клипы в `queued`, очищает `error`, `started_at`, `rendered_at`, `temp_path`, удаляет оставшийся `temp_path` с диска. Если `output_path` уже существует, предупреждает пользователя, а `clip` в очередь не возвращает.

Интеграция mpv и Lua

Lua-скрипт

Lua-скрипт должен реализовать горячие клавиши и писать каждое событие в JSONL-файл, путь к которому передаётся в опции `--script-opts=sf-marks-file=<session_file>`. Каждая запись должна содержать тип события, временные отметки и время записи. События следует сразу flush-ить, чтобы их не потерять при сбое.

Пример событий:

```
{ "event": "set_in", "pos": 123.4, "ts": "2026-06-04T12:00:00" }
{ "event": "mark", "in": 123.4, "out":
183.4, "rating": null, "label": null, "ts": "2026-06-04T12:00:10" }
{ "event": "quick_clip", "in": 250.0, "out":
```

```
310.0,"rating":null,"label":null,"ts":"2026-06-04T12:02:00"}
{"event":"done","ts":"2026-06-04T12:05:00"}
```

Каждый JSONL-файл должен называться по шаблону `sessions/<video_id>_<timestamp>_<random>.jsonl`. После успешного импорта файл не удаляется, что позволяет повторно импортировать его idempotently.

Горячие клавиши

Клавиша	Действие
i	отметить начало фрагмента (<code>event=set_in</code>)
o	отметить конец и сохранить фрагмент (<code>event=mark</code>)
s	быстро сохранить клип: текущая позиция до +60 сек, ограничивая по окончанию видео (<code>event=quick_clip</code>)
u	отменить последнюю отметку текущей сессии
d	завершить просмотр: пометить как reviewed (<code>event=done</code>)
n	пропустить видео (<code>event=skip</code>)
q	выйти без финального статуса (<code>event=quit</code>)

Импорт JSONL

1. `mpv_session.py` создаёт `review_session` со статусом `open` заранее и передаёт путь `session_file` в `mpv`.
2. После закрытия `mpv` статус сессии переводится в `closed`.
3. Python читает JSONL, импортирует events в базу:
4. все `mark` / `quick_clip` записываются в таблицу `marks`;
5. для каждого создаётся запись в `clips` со статусом `queued`.
6. В зависимости от финального события:
7. `done` → видео получает статус `reviewed`;
8. `skip` → `skipped`;
9. `quit` или отсутствие `done/skip/quit` → `inbox` и выводится предупреждение; метки всё равно сохраняются;
10. если `mpv` не запустился или JSONL отсутствует → `review_session.status=failed`, `review_status=inbox`.
11. `review_session` переводится в `imported`.

Workflow

Типовой рабочий цикл:

1. Загрузить видео: `shortfarm add /путь/к/файлу.mp4`.
2. Проверить список: `shortfarm inbox`.
3. Запустить просмотр: `shortfarm review inbox`.
4. Разметить моменты в mpv горячими клавишами, затем завершить (`d`), пропустить (`n`) или выйти (`q`).
5. После выхода Python импортирует JSONL, создаёт метки и клипы.
6. Посмотреть очередь клипов: `shortfarm clips --status queued`.
7. Запустить рендер: `shortfarm render --limit 5`.
8. Проверить готовые клипы: `shortfarm clips --status done`.
9. Повторять со следующими видео.

Атомарный выбор видео

Чтобы избежать ситуации, когда два терминала одновременно забирают одно и то же видео, следует выполнять выбор видео и перевод статуса в `reviewing` в рамках одной транзакции с блокировкой:

```
BEGIN IMMEDIATE;
SELECT id FROM videos WHERE review_status='inbox' ORDER BY id LIMIT 1;
UPDATE videos
  SET review_status='reviewing'
  WHERE id=? AND review_status='inbox';
COMMIT;
```

Если `UPDATE` изменил 0 строк, значит кто-то другой уже взял видео; CLI должна вывести “No inbox videos available”. При `rowcount=0` используется `COMMIT`; откат (`ROLLBACK`) не нужен, потому что изменений нет.

Обработка сценариев

Существуют различные типы завершения mpv-сессии:

Нормальное завершение

mpv завершается, JSONL существует и содержит событие `done` или `skip`. События импортируются, `review_session` получает статус `imported`, видео — `reviewed` или `skipped`.

Пользователь вышел (`q`)

mpv записывает `{"event": "quit"}`. Python импортирует метки и клипы, статус видео возвращается в `inbox`. Пользователь может продолжить просмотр позже; ранее созданные метки остаются.

mpv упал после запуска

Если `mpv` не стартовал: `session_file` не создан → `review_session.status=failed`, видео → `inbox`. Если `mpv` работал, но завершился без финального события: JSONL содержит метки, но нет `done/skip/quit` → импортировать метки, вернуть видео в `inbox`, отметить сессию как `imported` и записать предупреждение.

Ошибка импорта

Если JSONL не удаётся распарсить (битые строки), Python должен пропустить ошибочные записи, импортировать корректные, пометить `review_session.status=imported` и записать предупреждение в `review_session.error`. Фатальная ошибка (например, отсутствует файл) переводит сессию в `failed`, а видео — обратно в `inbox`.

Поведение `--force` и сброса

- Флаг `--force` разрешает повторный просмотр видео со статусами `reviewed`, `skipped` или `failed`. Он переводит `review_status` в `reviewing` и создаёт новую `review_session`, не удаляя старые метки.
- Для видео в статусе `reviewing` `--force` не применяется, чтобы не перезаписать текущую активную сессию. Пользователь должен явно выполнить `shortfarm review reset <video_id>` (или `--abandon-open-session`), чтобы пометить открытую сессию как `abandoned` и вернуть видео в `inbox`.

Тестирование и инфраструктура

- Все unit-тесты писать через `pytest`.
- Тесты должны использовать временную SQLite-базу (in-memory или `tmp_path`), применять миграции и заполнять таблицы.
- CLI-тесты выполнять через `Typer.CliRunner`, а не через `subprocess`.
- В тестах для рендера мокировать вызовы `ffmpeg/ffprobe` (использовать `monkeypatch` на `subprocess.run`), проверять аргументы и смену статусов, не запускать реальный `ffmpeg`.
- Предусмотреть fixtures в `tests/conftest.py` для создания базы, добавления видео, создания сессий, чтения JSONL.

Дополнительные уточнения

- COMMIT при `rowcount=0`.** В атомарном выборе видео при `UPDATE` изменившем 0 строк всегда нужно завершать транзакцию `COMMIT`, а не `ROLLBACK`. Откат здесь не нужен, так как данные не менялись.
- Сценарии аварийного закрытия.** Разделяются два случая:
- `mpv` не стартовал вообще → `session_file` отсутствует → `review_session.status=failed`, видео → `inbox`.
- `mpv` запустился, JSONL есть, но нет `done/skip/quit` → импортировать метки, вернуть видео в `inbox`, отметить сессию как `imported` и записать предупреждение.

5. **Поведение** `q`. Событие `quit` означает выход без решения. Метки сохраняются, видео возвращается в `inbox`. Это позволяет сохранить уже отмеченные фрагменты и продолжить работу позднее.
6. **Старые сессии при reset**. Команда `review reset <video_id>` переводит все открытые `review_sessions` для видео в статус `abandoned`, записывает `finished_at` и ошибку "Abandoned by user reset", затем возвращает видео в `inbox`.
7. **Список статусов** `review_sessions`. Используются статусы `open`, `closed`, `imported`, `failed`, `abandoned`. Состояние `closed` является промежуточным: mpv завершился, но импорт ещё не выполнен.

Заключение

Документ описывает подробный план внедрения MPV CLI workflow в проект ShortFarm. Он охватывает вопросы архитектуры, работы с базой, интеграции mpv и Lua, структуры CLI, обработки ошибок и тестирования. Реализация должна выполняться поэтапно: сначала исправления и миграции, затем импорт JSONL и mpv-интеграция, потом очередь клипов и рендер. Такой подход позволит сохранить работоспособность текущего функционала и постепенно расширить возможности инструмента.
